be able to tell that they actually refer to the same resource. An alternative is to pass the token as a query parameter, in which case the Dropbox URI would look like the following:

```
https://dl.dropboxusercontent.com/1/view/
[CA]book/graphics/CH08_FIG8.2_RBAC.png?token=8ygmwuqzf1l6x7c
```

There is a standard form for such URIs when the token is an OAuth2 token defined by RFC 6750 (https://tools.ietf.org/html/rfc6750#section-2.3) using the parameter name `access_token`. This is often the simplest approach to implement because it requires no changes to existing resources, but it shares some security weaknesses with the path-based approach:

- Both URI paths and query parameters are frequently logged by web servers and proxies, which can make the capability available to anybody who has access to the logs. Using TLS will prevent proxies from seeing the URI, but a request may still pass through several servers unencrypted in a typical deployment.
- The full URI may be visible to third parties through the HTTP `Referer` header or the `window.referrer` variable exposed to content running in an HTML iframe. You can use the `Referrer-Policy` header and `rel="noreferrer"` attribute on links in your UI to prevent this leakage. See https://developer.mozilla.org/en-US/docs/Web/Security/Referer_header:_privacy_and_security_concerns for details.